Hockey Cup Registration System

Mini Project Report

GIK29B –Python- och R-programming

Author: [Aayush Thapa Magar]

Date: 2026-03-16

Table of Contents

1. Introduction 1

2. Description of the Solution 2

2.1 team.py → Team 2

2.2 cup.py → Cup 2

2.3 team_repository → TeamRepository 2

2.4 team_storage.py → TeamStorage 3

2.5 user_interface.py → UserInterface 3

2.6 program.py → Entry Point 4

3. User Guide 4

3.1 How to Run the Program 4

3.2 Menu Options 5

3.3 Data Persistence 6

3.4 Participation Fee 6

1. Introduction

This project involves the design and implementation of an object-oriented Python program for managing hockey teams participating in a cup tournament. The system provides a command-line menu interface that allows users to register teams, update their information, track fee payments, handle cancellations and view statistics about all registered teams.

The program was developed for the GIK28B course at Dalarna University. It covers all Basic Functionality (G level) and (VG level) requirements, including file persistence, advanced search/filter options, statistics reporting and auto-save functionality.

The project does not handle match schedules or results. It focuses entirely on team administration: creating teams, keeping their data up to date, and persisting that data between sessions using a JSON file.

2. Description of the Solution

The system is split across six Python files, each containing exactly one class (or the program entry point). This follows the object-oriented principle of separation of concerns.

2.1 team.py → Team

Purpose: Represents a single hockey team. It is the core data model of the entire application.

Responsibilities:

- Stores all private team attributes: id, date_created, name, type, fee_paid, cancelled, cancellation_date.
- Expose a class-level constant PARTICIPATION_FEE (500 SEK) shared by all teams.
- Provides getters and setters for controlled access to private data (encapsulation).
- Provides cancel() and uncancel() methods for cancellation handling.
- Provides to_dict() and form_dict() class-method for JSON serializations and deserialization.

Collaborations: Used by TeamRepository to create and modify teams. Used by TeamStorage for serialization. Used by UserInterface to display team data.

2.2 cup.py → Cup

Purpose: Holds metadata about the hockey tournament itself (name, location, sport type, dates, description).

Responsibilities:

- Stores all private cup attributes with getters.
- Provides a formatted __str__ display for the cup information menu option.

Collaborations: Created in program.py and passed to UserInterface for display.

2.3 team_repository → TeamRepository

Purpose: Acts as the in-memory data store for all Team object.

Responsibilities:

- Stores teams in a dictionary (id -> Team) for O(1) lookup by ID.
- Manges a running __next_id counter that persists uniqueness even after loading form files.
- Provides full CRUD: add_team, get_team_by_id, get_all_teams, get_teams_by_type, update_team, cancel_team, delete_team.
- Provides search_by_name and filter_by_payment for search/filter functionality.
- Provides get_statistics() returning total teams, paid/unpaid counts, percentage paid and cancelled count.

Collaboration: Creates Team objects. Called by UserInterface for all data operations. Provides team lists to TeamStorage for saving.

2.4 team_storage.py → TeamStorage

Purpose: Handles all file I/O. this is the storage class.

Responsibilities:

- Saves a list of Team objects to a JSON file using to_dict() serialization.
- Loads Team object from a JSON file using Team.form_dict().
- Provides export_csv() as a bonus feature to export all team data to a .csv file.

Collaborations: Called by UserInterface after every mutating operation (auto-save). Collaborates with Team for serialization.

2.5 user_interface.py → UserInterface

Purpose: The command-line interface layer. It handles all user input and output.

Responsibilities:

- Displays the main menu and reads the user's choice.
- Calls the appropriate TeamRepository method for each menu option.
- Triggers __autosave() via TeamStorage after every add, update, cancel, or delete operation.

- Loads saved data form file at startup.

Collaborations: Depends on TeamRepository, TeamStorage, Cup and Team (for PARTICIPATON_FEE display).

2.6 program.py → Entry Point

Purpose: The entry point of the application. Contains main() and the if __name__ == "__main__" guard.

Responsibilities:

- Instantiates Cup, TeamRepository, TeamStorage and UserInterface.
- Calls ui.run() to start the interactive loop.

Collaborations: Creates and wires together all other classes.

3. User Guide

3.1 How to Run the Program

Prerequisites: Python 3.8 or later must be installed

All six .py files must be in the same folder:

team.py

cup.py

team_repository.py

team_storage.py

team_interface.py

user_interface.py

program.py

To start the program, open a terminal in the folder containing the files and run:

python3program.py

On first launch no data files exists yet, so the program starts with an empty team list. On subsequent launches it automatically loads teams form teams_data.json.

3.2 Menu Options

After starting, the following menu is displayed:

```
PS D:\Python-Project\Practice_project1> python3 .\program.py

  [Info] Loaded 3 team(s) from file.

=======================================================
 Dalarna Winter Cup 2026  —  Hockey Cup Manager
=======================================================
  1. Add new team
  2. View team by ID
  3. List all teams
  4. List boys teams
  5. List girls teams
  6. Update a team
  7. Cancel a team
  8. Delete a team
  9. Search teams by name
 10. Filter teams by payment status
 11. Show statistics
 12. Show cup information
 13. Export teams to CSV
  0. Exit
-------------------------------------------------------
  Your choice:
```

Option 1 -> Add new team: Enter the team name, type (boys/girls), and whether the fee has been paid. The ID and registration date are assigned automatically.

Option 2 -> View team by ID: Enter the team's ID to display all its details.

Option 3-5 -> List teams: Show all teams, only boys teams, or only girls teams.

Option 6 -> Update: Enter a team ID, then choose to update the name, gender type, or fee paid status. ID and registration date cannot be changed.

Option 7 -> Cancel: Marks a team as cancelled and records today as the cancellation date (YYYY-MM-DD).

Option 8 -> Delete: Permanently removes a team from the system after confirmation.

Option 9 -> Search by name: Enter a keyword; all teams whose names contain it (case-insensitive) are shown.

Option 10 -> Filter by payment: Choose y (paid) or n (unpaid) to filter the team list.

Option 11 -> Statistic: Shows total teams, number and percentage that paid, number unpaid, number cancelled and fee totals.

Option 12 -> Cup information: Displays the tournament name, location, sport types, dates and description.

Option 13 -> Export to CSV: Saves all team data to a .csv file. You can enter a filename or press Enter to use the default (teams_export.csv).

Option 0 -> Exit: Cleanly exits the program.

3.3 Data Persistence

The program automatically saves data to teams __data.json after every add, update, cancel or delete operation. You do not need to save manually. Teams IDs remains unique and stable across sessions – a team saved with ID 23 will always have ID 23 when the file is reloaded.

3.4 Participation Fee

The participation fee (500 SEK) is stored as a class variable on the Team class. This means the same fee value applies to all teams and can be updated in one place if the fee changes. It is distinct form the fee_paid attribute, which only tracks weather a specific team has paid.